Parallel PageRank Algorithm Summary

Overview

PageRank ranks nodes in a graph based on incoming links. Each node distributes its rank to neighbors. The algorithm iterates until ranks converge.

Core Formula

NewRank(i) = (1 - d)/N + d * sum(Rank(j) / OutDegree(j)) for all j pointing to i Where d is the damping factor (typically 0.85).

Core Algorithm (Per Iteration)

1. Initialize all node ranks = 1/N 2. For each node i: - Collect contributions from incoming neighbors - Apply PageRank formula 3. Compute difference between old and new ranks 4. Repeat until convergence or max iterations

Sequential Version

Compute rank updates using a loop over all nodes. Use adjacency list or CSR for efficiency.

OpenMP Version

Parallelize loop over nodes. Use reductions for convergence calculation. Ensure no race conditions when updating ranks.

MPI Version

Partition nodes across processes. Exchange rank values between processes each iteration. Use MPI_Allgather or MPI_Allreduce for communication and convergence.

Key Challenges

- Load balancing graph partitions - Communication overhead between processes - Global convergence detection - Efficient sparse graph representation

Key Idea

Each node distributes its rank to neighbors, then updates its own rank based on incoming contributions.

N-Body Simulation Algorithm Summary

Overview

The N-Body problem simulates the motion of particles interacting under gravitational forces. Each particle affects every other particle, leading to O(N^2) computations per timestep.

Core Algorithm (Per Timestep)

1. For each particle i: - Initialize total force to zero. 2. For each other particle j: - Compute gravitational force between i and j. - Add this force to particle i. 3. Update motion: - Acceleration = Force / Mass - Velocity = Velocity + Acceleration × dt - Position = Position + Velocity × dt 4. Repeat for all particles and timesteps.

Sequential Version

Run the full double loop over all particles (i and j). No parallelism.

OpenMP Version

Parallelize the outer loop over particles. Use reductions to safely accumulate forces. Ensure thread-safe updates.

MPI Version

Distribute particles across processes. Use MPI_Allgather to share particle positions each timestep. Synchronize all processes before updating positions.

Key Challenges

- High complexity: O(N^2) - Communication overhead in MPI - Synchronization between processes - Avoiding race conditions in OpenMP

Key Idea

For every particle, compute forces from all others, then update velocity and position.

Standardized Pseudocode Templates (PageRank & N-Body)

Unified Structure

```
Initialize data
For iter = 1 → max_iterations:
    Compute intermediate values
    Update state
    Synchronize (if needed)
    Check convergence (optional)
End
```

PageRank - Sequential

```
Initialize:
    rank[i] = 1 / N

For iter = 1 → max_iterations:
    For each node i:
        new_rank[i] = (1 - d) / N

    For each node j:
        For each neighbor i of j:
            contribution = rank[j] / out_degree[j]
            new_rank[i] += d * contribution

    diff = 0
    For each node i:
        diff += |new_rank[i] - rank[i]|
        rank[i] = new_rank[i]

    If diff < tolerance:
        Break
End
```

PageRank - OpenMP

```
Parallelize loops over nodes
Use reduction for diff
```

PageRank - MPI

```
Distribute nodes across processes
Exchange rank using MPI_Allgather
Compute local updates
Use MPI_Allreduce for diff
```

N-Body - Sequential

```
For iter = 1 → max_iterations:
    For each particle i:
        force[i] = 0

    For each particle i:
        For each particle j ≠ i:
            compute force
            accumulate force[i]
```

```
    For each particle i:
        update velocity and position
End
```

N-Body - OpenMP

```
Parallelize outer loop over particles
Each thread computes force[i] independently
```

N-Body - MPI

```
Distribute particles
Exchange positions using MPI_Allgather
Compute local forces
Synchronize before updates
```